

```

1 # функция для рисования разделяющей поверхности на плоскости
2 # Div_plate(clf,flip_level) принимает объект-классификатор и уровень
3 # создает двумерные данные, обучает классификатор, рисует разделяющую
4 # Классификация с шумом
5
6 def Div_plate(clf, flip_level):
7     from sklearn.datasets import make_classification # для генерации
8     from sklearn.model_selection import train_test_split # для раздел
9     import numpy as np
10    import seaborn as sns # для простого отображения
11    import matplotlib.pyplot as plt #
12    # создаем данные
13    X,y=make_classification(n_samples=10000, # число примеров
14                            n_features=2, # число признаков (атрибутов)
15                            n_informative=1, # из них информативных
16                            n_redundant=0, # из них не информативных
17                            n_repeated=0, # из них повторяющихся
18                            n_classes=2, # число классов
19                            n_clusters_per_class=1, # число кластеров
20                            class_sep=1, # влияет на расстояние между
21                            flip_y=flip_level, # доля ошибок (шума)
22                            weights=[0.4,0.6], # пропорции числа данн
23                            random_state=17) #
24    # разделяем на обучающие и тестовые, случайно
25    X_train, X_test, y_train, y_test=train_test_split(
26        X, y, test_size=0.33, random_state=42)
27    # рисуем данные
28    plt.subplots();#
29    ax1=plt.gca();#
30    sns.scatterplot(x=X_train[:,0],y=X_train[:,1],hue=y_train,ax=ax1)
31    sns.scatterplot(x=X_test[:,0],y=X_test[:,1],hue=y_test,ax=ax1,mar
32    ax1.set_title("Data With Noise");#
33    plt.show();#
34    # Создаем классификатор на основе случайного леса. Изменяйте пара
35    clf.fit(X_train, y_train) # обучаем
36    y_pred=clf.predict(X_test) # проверяем на тестовых данных
37    score=clf.score(X_test, y_test) # считаем среднюю точность
38    print('Score=',score)
39    ind=y_test==y_pred; # индексы совпадений результата классификации
40    plt.subplots();
41    ax2=plt.gca();
42    # рисуем "правильно" распознанные примеры
43    sns.scatterplot(x=X_test[ind,0],y=X_test[ind,1],hue=y_test[ind],ax
44    # рисуем "неправильно" распознанные примеры
45    sns.scatterplot(x=X_test[~ind,0],y=X_test[~ind,1],hue=y_pred[~inc
46    #sns.scatterplot(X_test[:,0],X_test[:,1],hue=y_test[:,],ax=ax2,mar
47    ax2.set_title("With Noise");
48    # считаем и рисуем разделяющую поверхность.
49    plot_step=0.01
50    x_min, x_max=X[:, 0].min()-1, X[:, 0].max()+1 # немного измененны

```

```

51 y_min, y_max=X[:, 1].min()-1, X[:, 1].max()+1 # немного измененны
52 # считаем прямоугольную сетку возможных значений этих атрибутов
53 xx, yy=np.meshgrid(np.arange(x_min, x_max, plot_step), #
54                     np.arange(y_min, y_max, plot_step)) #
55 # считаем выход классификатора для всех примеров сетки
56 # не забыв что массивы данных нужно привести к требуемому размеру
57 Z=clf.predict(np.c_[xx.ravel(), yy.ravel()])
58 Z=Z.reshape(xx.shape) # и преобразуем обратно в исходному размеру
59 # рисуем разделяющую поверхность
60 cs=plt.contourf(xx, yy, Z, levels=1, colors=['blue','orange'],alpha=0.5)
61 plt.show();
62
63 from sklearn.ensemble import StackingClassifier #
64 from sklearn.svm import SVC #
65 from sklearn.linear_model import LogisticRegression #
66 from sklearn.tree import DecisionTreeClassifier
67 estimators=[('SVC', SVC(max_iter=10)),
68             ('LogReg', LogisticRegression()),
69             ('Tree', DecisionTreeClassifier(max_depth=5))]
70 clf=StackingClassifier(estimators=estimators)
71 flip_level=0.4
72 Div_plate(clf, flip_level)

```



```

/usr/local/lib/python3.12/dist-packages/sklearn/svm/_base.py:305: Conve
warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/svm/_base.py:305: Conve
warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/svm/_base.py:305: Conve
warnings.warn(

```

```
/usr/local/lib/python3.12/dist-packages/sklearn/svm/_base.py:305: Conve
warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/svm/_base.py:305: Conve
warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/svm/_base.py:305: Conve
warnings.warn(
Score= 0.7627272727272727
```



